Date: -27/10/26

CO-2 Assessment Tool-3

Error Analysis

Course Code: DSA 0308

Course Name: Natural Language Processing

Name: Monika R **Reg no: 192324281**

Question 1: Morphological Analysis Error in Biomedical Dataset

Aim

To identify errors caused by applying Porter Stemming directly to biomedical words and explain why inflectional and derivational morphology should be treated differently.

Error program:

```
from nltk.stem import PorterStemmer

words = ["infection", "infectious", "infected", "infect"]

for word in words:
    print(word, "->", word)
```

Output:

```
>>>
    === RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/2.3.7.py ===
    infection -> infection
    infectious -> infectious
    infected -> infected
    infect -> infect
>>>
```

The program prints the original word instead of applying stemming.

Corrected program:

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
words = ["infection", "infectious", "infected", "infect"]

for word in words:
    print(word, "->", stemmer.stem(word))
```

Output:

```
>>>
    === RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/2.3.7.py ===
    infection -> infect
    infectious -> infecti
    infected -> infect
    infect -> infect
>>>
```

Correction: stemmer.stem(word) applies the Porter stemming algorithm to each word.

Error approach

The incorrect approach is to stem every word immediately and use the resulting stems as complete morphological analyses.

Error analysis

Word	**Morphological information**	**Issue**
infection	Noun related to infect	The ending -ion carries derivational information
infectious	Adjective related to infection	The ending -ious indicates an adjective
infected	Past participle or inflected verb form	The ending -ed can indicate inflection
infect	Base verb	The original form should be preserved

Inflectional vs. derivational morphology

- Inflectional morphology changes grammatical form without normally changing the word's basic lexical category. Example: infect → infected.
- Derivational morphology creates a related word and may change its grammatical category or meaning. Example: infect → infection or infectious.

Result: The error analysis demonstrates that Porter stemming alone is insufficient for detailed biomedical morphological analysis. Retaining original words and distinguishing inflectional from derivational morphology can help preserve clinically important information.

Q2. Finite-State Transducer (FST) for social media words

Aim

To identify why a finite-state morphological parser fails to recognize words containing multiple prefixes and suffixes and demonstrate a corrected rule-based parser.

Error program:

```
words = ["unbelievable", "playing", "happily"]

for word in words:
    if word == "unbelievable":
        print(word, "->", "believe")
    elif word == "playing":
        print(word, "->", "play")
    else:
        print(word, "->", word)
```

Output:

```
>>>
    === RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/2.3.7.py ===
    unbelievable -> believe
    playing -> play
    happily -> happily
>>>
```

The program uses hard-coded conditions and does not implement a general FST or process suffixes consistently.

Error approach

A parser that handles only one affix or checks suffixes without considering prefix combinations may fail on words such as happiest, unbelievable, running, reordering, smartphones, and unreadable.

Corrected program:

```
def fst_parser(word):
    word = word.lower()

    if word.startswith("un") and len(word) > 2:
        word = word[2:]

    if word.endswith("ing") and len(word) > 3:
        word = word[:-3]
    elif word.endswith("ly") and len(word) > 2:
        word = word[:-2]

    return word

words = ["unhappy", "playing", "quickly"]

for word in words:
    print(word, "->", fst_parser(word))
```

Output:

```
>>
=== RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/2.3.7.py ===
unhappy -> happy
playing -> play
quickly -> quick
>>
```

Why the parser fails

A finite-state transducer uses states and transitions to recognize valid combinations of morphemes. If the transitions support only one affix, words containing multiple affixes may not be analyzed correctly.

The corrected demonstration applies prefix and suffix rules sequentially. A complete FST should additionally handle spelling changes, irregular forms, and invalid combinations.

Result: The corrected demonstration recognizes common prefix and suffix combinations. Further FST rules are required to achieve reliable morphological analysis and measure accuracy on the Sentiment 140 dataset.

QUESTION 6 – Error Analysis of a Finite-State Morphological Parser

Aim

To identify logical errors in a plural noun parser and correct it to handle regular plurals, -es endings, -ies endings, and irregular plural nouns.

Error program

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = ["cars", "boxes", "cities", "children"]

for w in words:
    print(parser(w))
```

Output:

```
=== RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/2.3.7.py ===
('ca', 'Plural Noun')
('box', 'Plural Noun')
('citi', 'Plural Noun')
('children', 'Singular')
>>
```

Why it is wrong: Removing only the final s fails for words such as boxes, cities, and irregular plurals such as children.

Error Approach:

The program incorrectly removes two characters from every word ending in s. It does not handle regular plurals, -es endings, -ies endings, or irregular plurals correctly. For example, cars becomes ca, cities is not converted to city, and children is incorrectly classified as singular.

Corrected program:

```
irregular = {
    "children": "child",
    "men": "man",
    "women": "woman",
    "mice": "mouse",
    "feet": "foot",
    "teeth": "tooth",
    "people": "person"
}

def parser(word):
    word = word.lower()

    if word in irregular:
        return word, irregular[word], "Plural Noun"

    elif word.endswith("ies") and len(word) > 3:
        return word, word[:-3] + "y", "Plural Noun"

    elif word.endswith(("ches", "shes", "sses", "xes", "zes")):
        return word, word[:-2], "Plural Noun"

    elif word.endswith("s") and not word.endswith("ss"):
        return word, word[:-1], "Plural Noun"

    else:
        return word, word, "Singular"

words = ["cars", "boxes", "cities", "children", "dogs", "book"]

for word in words:
    original, base, category = parser(word)
    print(original, "->", base, "-", category)
```

Output:

```
=== RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/2.3.7.py ===
cars -> car - Plural Noun
boxes -> box - Plural Noun
cities -> city - Plural Noun
children -> child - Plural Noun
dogs -> dog - Plural Noun
book -> book - Singular
>
```

Explanation of the correction

Plural rule	Example	Correct base
Regular plural -s	cars	car
Plural -es	boxes	box
Plural -ies	cities	city
Irregular plural	children	child
Singular noun	book	book

The parser first checks irregular forms, then applies specific suffix rules before the general -s rule. This order prevents a general rule from incorrectly processing words that require special treatment.

Result:

The corrected parser handles the required common plural patterns and avoids the original error of removing two characters from every word ending in s.

QUESTION 7 – Error Analysis of Morphological Feature Extraction

Aim

To identify errors caused by applying Porter stemming after feature extraction and demonstrate a corrected preprocessing pipeline.

Error program

```
import re
from nltk.stem import PorterStemmer

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]

for document in documents:
    words = re.findall(r"[a-z]+", document.lower())
    print(stemmer.stem(words))
```

Output:

```
>
= RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/22222222222222
222.py
Traceback (most recent call last):
  File "C:/Users/monik/AppData/Local/Programs/Python/Python314/22222222222222222
.py", line 12, in <module>
    print(stemmer.stem(words))
NameError: name 'stemmer' is not defined
>
```

Error Approach:

The program applies Porter Stemming after feature extraction using CountVectorizer. Consequently, related word forms may retain separate columns in the original feature matrix even when their feature names are converted to the same stem. This creates redundant vocabulary and inconsistency between the feature names and the feature matrix. Stemming should be applied before feature extraction.

Corrected program:

```
import re
import time
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]

start = time.time()

original_words = []
stemmed_words = []

for document in documents:
    words = re.findall(r"[a-z]+", document.lower())
    original_words.extend(words)

    for word in words:
        stemmed_words.append(stemmer.stem(word))

original_vocab = set(original_words)
stemmed_vocab = set(stemmed_words)

end = time.time()

print("Original vocabulary size:", len(original_vocab))
print("Stemmed vocabulary size:", len(stemmed_vocab))
print("Original vocabulary:", sorted(original_vocab))
print("Stemmed vocabulary:", sorted(stemmed_vocab))
print("Processing time:", round(end - start, 6), "seconds")
```

Output:

```
= RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/2222222222222222.p
y
Original vocabulary size: 9
Stemmed vocabulary size: 4
Original vocabulary: ['organization', 'organized', 'organizer', 'runners', 'running',
'runs', 'studied', 'studies', 'studying']
Stemmed vocabulary: ['organ', 'run', 'runner', 'studi']
Processing time: 0.000193 seconds
>
```

Effect on NLP performance

- Vocabulary size: Stemming can reduce the number of distinct features.
- Search and information retrieval: Related word forms may match more easily.
- Processing time: Stemming adds preprocessing work, but a smaller vocabulary can reduce later feature-processing costs.
- Classification accuracy: It may improve or reduce accuracy depending on the dataset. Accuracy must be measured using a trained classifier; vocabulary reduction alone does not prove improved accuracy.

Result: Stemming should be performed before feature extraction so that the feature matrix is built from normalized words rather than retaining separate columns for related word forms.